

第 7 章

使用 RAID 与 LVM 磁盘阵列技术

本章讲解了如下内容：

- RAID（独立冗余磁盘阵列）；
- LVM（逻辑卷管理器）。

在学习了第 6 章讲解的硬盘设备分区、格式化、挂载等知识后，本章将深入讲解各个常用 RAID（Redundant Array of Independent Disks，独立冗余磁盘阵列）技术方案的特性，并通过实际部署 RAID 10、RAID 5+备份盘等方案来更直观地查看 RAID 的强大效果，以便进一步满足生产环境对硬盘设备的 IO 读写速度和数据冗余备份机制的需求。同时，考虑到用户可能会动态调整存储资源，本章还将介绍 LVM（Logical Volume Manager，逻辑卷管理器）的部署、扩容、缩小、快照以及卸载删除的相关知识。相信读者在学完本章内容后，便可以在企业级生产环境中灵活运用 RAID 和 LVM 来满足对存储资源的高级管理需求了。

7.1 RAID（独立冗余磁盘阵列）

近年来，CPU 的处理性能保持着高速增长，Intel 公司在 2017 年最新发布的 i9-7980XE 处理器芯片更是达到了 18 核心 36 线程。但与此同时，硬盘设备的性能提升却不是很大，因此逐渐成为当代计算机整体性能的瓶颈。而且，由于硬盘设备需要进行持续、频繁、大量的 IO 操作，相较于其他设备，其损坏几率也大幅增加，导致重要数据丢失的几率也随之增加。

1988 年，加利福尼亚大学伯克利分校首次提出并定义了 RAID 技术的概念。RAID 技术通过把多个硬盘设备组合成一个容量更大、安全性更好的磁盘阵列，并把数据切割成多个区段后分别存放在各个不同的物理硬盘设备上，然后利用分散读写技术来提升磁盘阵列整体的性能，同时把多个重要数据的副本同步到不同的物理硬盘设备上，从而起到了非常好的数据冗余备份效果。

任何事物都有它的两面性。RAID 技术确实具有非常好的数据冗余备份功能，但是它也相应地提高了成本支出。就像原本我们只有一个电话本，但是为了避免遗失，我们将联系人号码信息写成了两份，自然要为此多买一个电话本，这也就相应地提升了成本支出。RAID 技术的设计初衷是减少因为采购硬盘设备带来的费用支出，但是与数据本身的价值相比较，现代企业更看重的则是 RAID 技术所具备的冗余备份机制以及带来的硬盘吞吐量的提升。也就是说，RAID 不仅降低了硬盘设备损坏后丢失数据的几率，还提升了硬盘设备的读写速度，所以它在绝大多数运营商或大中型企业中得以广泛部署和应用。

出于成本和技术方面的考虑，需要针对不同的需求在数据可靠性及读写性能上作出权衡，制定出满足各自需求的不同方案。目前已有的 RAID 磁盘阵列的方案至少有十几种，而刘遛

老师接下来会详细讲解 RAID 0、RAID 1、RAID 5 与 RAID 10 这 4 种最常见的方案。

7.1.1 RAID 0

RAID 0 技术把多块物理硬盘设备（至少两块）通过硬件或软件的方式串联在一起，组成一个大的卷组，并将数据依次写入到各个物理硬盘中。这样一来，在最理想的状态下，硬盘设备的读写性能会提升数倍，但是若任意一块硬盘发生故障将导致整个系统的数据都受到破坏。通俗来说，RAID 0 技术能够有效地提升硬盘数据的吞吐速度，但是不具备数据备份和错误修复能力。如图 7-1 所示，数据被分别写入到不同的硬盘设备中，即 disk1 和 disk2 硬盘设备会分别保存数据资料，最终实现提升读取、写入速度的效果。

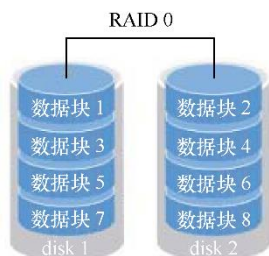


图 7-1 RAID 0 技术示意图

7.1.2 RAID 1

尽管 RAID 0 技术提升了硬盘设备的读写速度，但是它是将数据依次写入到各个物理硬盘中，也就是说，它的数据是分开存放的，其中任何一块硬盘发生故障都会损坏整个系统的数据。因此，如果生产环境对硬盘设备的读写速度没有要求，而是希望增加数据的安全性时，就需要用到 RAID 1 技术了。

在图 7-2 所示的 RAID 1 技术示意图中可以看到，它是把两块以上的硬盘设备进行绑定，在写入数据时，是将数据同时写入到多块硬盘设备上（可以将其视为数据的镜像或备份）。当其中某一块硬盘发生故障后，一般会立即自动以热交换的方式来恢复数据的正常使用。

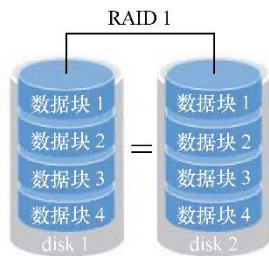


图 7-2 RAID 1 技术示意图

RAID 1 技术虽然十分注重数据的安全性，但是因为是在多块硬盘设备中写入了相同的数

据, 因此硬盘设备的利用率得以下降, 从理论上来说, 图 7-2 所示的硬盘空间的真实可用率只有 50%, 由三块硬盘设备组成的 RAID 1 磁盘阵列的可用率只有 33%左右, 以此类推。而且, 由于需要把数据同时写入到两块以上的硬盘设备, 这无疑也在一定程度上增大了系统计算功能的负载。

那么, 有没有一种 RAID 方案既考虑到了硬盘设备的读写速度和数据安全性, 还兼顾了成本问题呢? 实际上, 单从数据安全和成本问题上来讲, 就不可能在保持原有硬盘设备的利用率且还不增加新设备的情况下, 能大幅提升数据的安全性。刘遒老师也没有必要忽悠各位读者, 下面将要讲解的 RAID 5 技术虽然在理论上兼顾了三者(读写速度、数据安全性、成本), 但实际上更像是对这三者的“相互妥协”。

7.1.3 RAID 5

如图 7-3 所示, RAID5 技术是把硬盘设备的数据奇偶校验信息保存到其他硬盘设备中。RAID 5 磁盘阵列组中数据的奇偶校验信息并不是单独保存到某一块硬盘设备中, 而是存储到除自身以外的其他每一块硬盘设备上, 这样的好处是其中任一设备损坏后不至于出现致命缺陷; 图 7-3 中 parity 部分存放的就是数据的奇偶校验信息, 换句话说, 就是 RAID 5 技术实际上没有备份硬盘中的真实数据信息, 而是当硬盘设备出现问题后通过奇偶校验信息来尝试重建损坏的数据。RAID 这样的技术特性“妥协”地兼顾了硬盘设备的读写速度、数据安全性与存储成本问题。

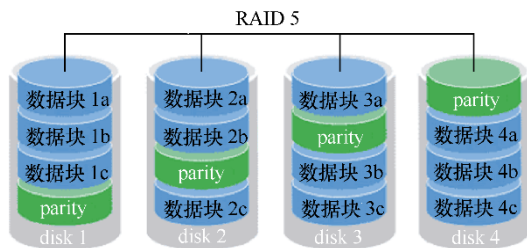


图 7-3 RAID5 技术示意图

7.1.4 RAID 10

鉴于 RAID 5 技术是因为硬盘设备的成本问题对读写速度和数据的安全性能而有了一定的妥协, 但是大部分企业更在乎的是数据本身的价值而非硬盘价格, 因此生产环境中主要使用 RAID 10 技术。

顾名思义, RAID 10 技术是 RAID 1+RAID 0 技术的一个“组合体”。如图 7-4 所示, RAID 10 技术需要至少 4 块硬盘来组建, 其中先分别两两制作成 RAID 1 磁盘阵列, 以保证数据的安全性; 然后再对两个 RAID 1 磁盘阵列实施 RAID 0 技术, 进一步提高硬盘设备的读写速度。这样从理论上讲, 只要坏的不是同一组中的所有硬盘, 那么最多可以损坏 50% 的硬盘设备而不丢失数据。由于 RAID 10 技术继承了 RAID 0 的高读写速度和 RAID 1 的数据安全性, 在不考虑成本的情况下 RAID 10 的性能都超过了 RAID 5, 因此当前成为广泛使用的一种存储技术。

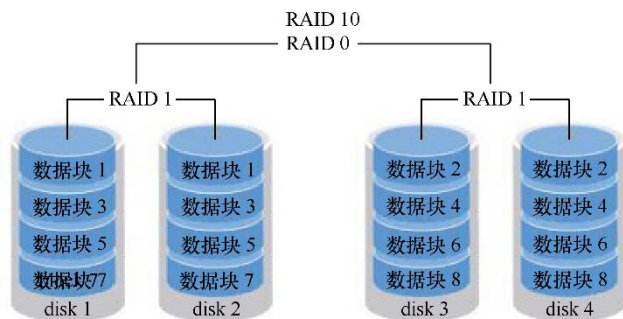


图 7-4 RAID 10 技术示意图

7.1.5 部署磁盘阵列

在具备了上一章的硬盘设备管理基础之后，再来部署 RAID 和 LVM 就变得十分轻松了。首先，需要在虚拟机中添加 4 块硬盘设备来制作一个 RAID 10 磁盘阵列，如图 7-5 所示。

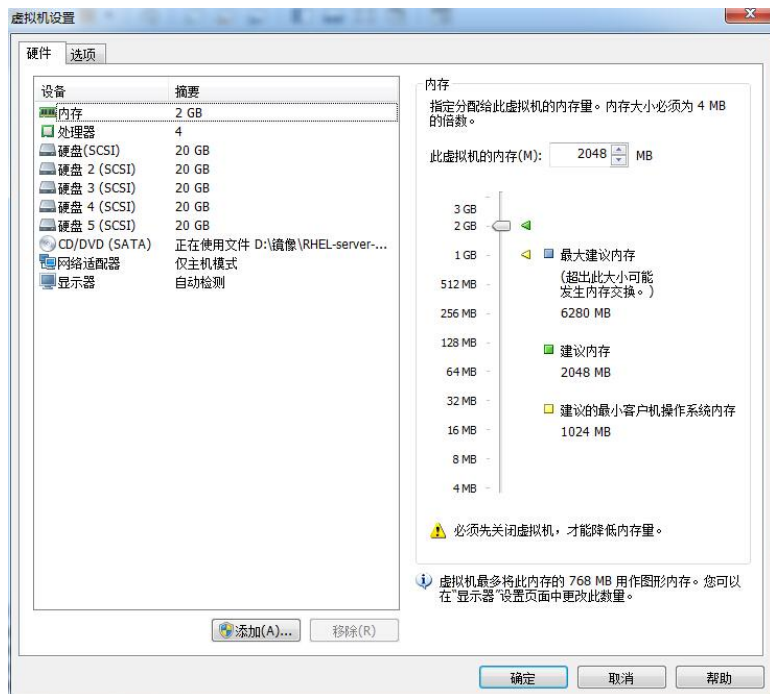


图 7-5 为虚拟机系统模拟添加 4 块硬盘设备

这几块硬盘设备是模拟出来的，不需要特意去买几块真实的物理硬盘插到电脑上。需要注意的是，一定要记得在关闭系统之后，再在虚拟机中添加硬盘设备，否则可能会因为计算机架构的不同而导致虚拟机系统无法识别添加的硬盘设备。

mdadm 命令用于管理 Linux 系统中的软件 RAID 硬盘阵列，格式为“mdadm [模式] <RAID 设备名称> [选项] [成员设备名称]”。

当前，生产环境中用到的服务器一般都配备 RAID 阵列卡，尽管服务器的价格越来越便宜，但是我们没有必要为了做一个实验而去单独购买一台服务器，而是可以学会用 mdadm 命

令在 Linux 系统中创建和管理软件 RAID 磁盘阵列，而且它涉及的理论知识的操作过程与生产环境中的完全一致。mdadm 命令的常用参数以及作用如表 7-1 所示。

表 7-1 mdadm 命令的常用参数和作用

参数	作用
-a	检测设备名称
-n	指定设备数量
-l	指定 RAID 级别
-C	创建
-v	显示过程
-f	模拟设备损坏
-r	移除设备
-Q	查看摘要信息
-D	查看详细信息
-S	停止 RAID 磁盘阵列

接下来，使用 mdadm 命令创建 RAID 10，名称为 “/dev/md0”。

第 6 章中讲到，udev 是 Linux 系统内核中用来给硬件命名的服务，其命名规则也非常简单。我们可以通过命名规则猜测到第二个 SCSI 存储设备的名称会是 /dev/sdb，然后依此类推。使用硬盘设备来部署 RAID 磁盘阵列很像是将几位同学组成一个班级，但总不能将班级命名为 /dev/sdbcde 吧。尽管这样可以一眼看出它是由哪些元素组成的，但是并不利于我们的记忆和阅读。更何况如果我们使用的是 10、50、100 个硬盘来部署 RAID 磁盘阵列呢？

此时，就需要使用 mdadm 中的参数了。其中，-C 参数代表创建一个 RAID 阵列卡；-v 参数显示创建的过程，同时在后面追加一个设备名称 /dev/md0，这样 /dev/md0 就是创建后的 RAID 磁盘阵列的名称；-a yes 参数代表自动创建设备文件；-n 4 参数代表使用 4 块硬盘来部署这个 RAID 磁盘阵列；而 -l 10 参数则代表 RAID 10 方案；最后再加上 4 块硬盘设备的名称就搞定了。

```
[root@linuxprobe ~]# mdadm -Cv /dev/md0 -a yes -n 4 -l 10 /dev/sdb /dev/sdc
/dev/sdd /dev/sde
mdadm: layout defaults to n2
mdadm: layout defaults to n2
mdadm: chunk size defaults to 512K
mdadm: size set to 20954624K
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

其次，把制作好的 RAID 磁盘阵列格式化为 ext4 格式。

```
[root@linuxprobe ~]# mkfs.ext4 /dev/md0
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=128 blocks, Stripe width=256 blocks
```

```
2621440 inodes, 10477312 blocks
523865 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=2157969408
320 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
4096000, 7962624
Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done
```

再次，创建挂载点然后把硬盘设备进行挂载操作。挂载成功后可看到可用空间为40GB。

```
[root@linuxprobe ~]# mkdir /RAID
[root@linuxprobe ~]# mount /dev/md0 /RAID
[root@linuxprobe ~]# df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/rhel-root	18G	3.0G	15G	17%	/
devtmpfs	905M	0	905M	0%	/dev
tmpfs	914M	84K	914M	1%	/dev/shm
tmpfs	914M	8.9M	905M	1%	/run
tmpfs	914M	0	914M	0%	/sys/fs/cgroup
/dev/sr0	3.5G	3.5G	0	100%	/media/cdrom
/dev/sda1	497M	119M	379M	24%	/boot
/dev/md0	40G	49M	38G	1%	/RAID

最后，查看/dev/md0 磁盘阵列的详细信息，并把挂载信息写入到配置文件中，使其永久生效。

```
[root@linuxprobe ~]# mdadm -D /dev/md0
/dev/md0:
Version : 1.2
Creation Time : Tue May 5 07:43:26 2017
Raid Level : raid10
Array Size : 41909248 (39.97 GiB 42.92 GB)
Used Dev Size : 20954624 (19.98 GiB 21.46 GB)
Raid Devices : 4
Total Devices : 4
Persistence : Superblock is persistent
Update Time : Tue May 5 07:46:59 2017
State : clean
Active Devices : 4
Working Devices : 4
Failed Devices : 0
Spare Devices : 0
Layout : near=2
Chunk Size : 512K
Name : localhost.localdomain:0 (local to host localhost.localdomain)
UUID : cc9a87d4:1e89e175:5383e1e8:a78ec62c
Events : 17
```

```
Number Major Minor RaidDevice State
0 8 16 0 active sync /dev/sdb
1 8 32 1 active sync /dev/sdc
2 8 48 2 active sync /dev/sdd
3 8 64 3 active sync /dev/sde
[root@linuxprobe ~]# echo "/dev/md0 /RAID ext4 defaults 0 0" >> /etc/fstab
```

7.1.6 损坏磁盘阵列及修复

之所以在生产环境中部署 RAID 10 磁盘阵列，是为了提高硬盘存储设备的读写速度及数据的安全性，但由于我们的硬盘设备是在虚拟机中模拟出来的，因此对读写速度的改善可能并不直观，因此刘遛老师决定给各位读者讲解一下 RAID 磁盘阵列损坏后的处理方法，这样大家在步入运维岗位后遇到类似问题时，也可以轻松解决。

在确认有一块物理硬盘设备出现损坏而不能继续正常使用后，应该使用 `mdadm` 命令将其移除，然后查看 RAID 磁盘阵列的状态，可以发现状态已经改变。

```
[root@linuxprobe ~]# mdadm /dev/md0 -f /dev/sdb
mdadm: set /dev/sdb faulty in /dev/md0
[root@linuxprobe ~]# mdadm -D /dev/md0
/dev/md0:
Version : 1.2
Creation Time : Fri May 8 08:11:00 2017
Raid Level : raid10
Array Size : 41909248 (39.97 GiB 42.92 GB)
Used Dev Size : 20954624 (19.98 GiB 21.46 GB)
Raid Devices : 4
Total Devices : 4
Persistence : Superblock is persistent
Update Time : Fri May 8 08:27:18 2017
State : clean, degraded
Active Devices : 3
Working Devices : 3
Failed Devices : 1
Spare Devices : 0
Layout : near=2
Chunk Size : 512K
Name : linuxprobe.com:0 (local to host linuxprobe.com)
UUID : f2993bbd:99c1eb63:bd61d4d4:3f06c3b0
Events : 21
Number Major Minor RaidDevice State
0 0 0 0 removed
1 8 32 1 active sync /dev/sdc
2 8 48 2 active sync /dev/sdd
3 8 64 3 active sync /dev/sde
0 8 16 - faulty /dev/sdb
```

在 RAID 10 级别的磁盘阵列中，当 RAID 1 磁盘阵列中存在一个故障盘时并不影响 RAID 10 磁盘阵列的使用。当购买了新的硬盘设备后再使用 `mdadm` 命令来予以替换即可，在此期间我们可以在 `/RAID` 目录中正常地创建或删除文件。由于我们是在虚拟机中模拟硬盘，所以先重启系统，然后再把新的硬盘添加到 RAID 磁盘阵列中。

```
[root@linuxprobe ~]# umount /RAID
```

```
[root@linuxprobe ~]# mdadm /dev/md0 -a /dev/sdb
[root@linuxprobe ~]# mdadm -D /dev/md0
/dev/md0:
  Version : 1.2
  Creation Time : Mon Jan 30 00:08:56 2017
  Raid Level : raid10
  Array Size : 41909248 (39.97 GiB 42.92 GB)
  Used Dev Size : 20954624 (19.98 GiB 21.46 GB)
  Raid Devices : 4
  Total Devices : 4
  Persistence : Superblock is persistent
  Update Time : Mon Jan 30 00:19:53 2017
  State : clean
  Active Devices : 4
  Working Devices : 4
  Failed Devices : 0
  Spare Devices : 0
  Layout : near=2
  Chunk Size : 512K
  Name : localhost.localdomain:0 (local to host localhost.localdomain)
  UUID : d3491c05:cfc81ca0:32489f04:716a2cf0
  Events : 56
Number Major Minor RaidDevice State
  4 8 16 0 active sync /dev/sdb
  1 8 32 1 active sync /dev/sdc
  2 8 48 2 active sync /dev/sdd
  3 8 64 3 active sync /dev/sde
[root@linuxprobe ~]# mount -a
```

7.1.7 磁盘阵列+备份盘

RAID 10 磁盘阵列中最多允许 50%的硬盘设备发生故障，但是存在这样一种极端情况，即同一 RAID 1 磁盘阵列中的硬盘设备若全部损坏，也会导致数据丢失。换句话说，在 RAID 10 磁盘阵列中，如果 RAID 1 中的某一块硬盘出现了故障，而我们正在前往修复的路上，恰巧该 RAID1 磁盘阵列中的另一块硬盘设备也出现故障，那么数据就被彻底丢失了。刘遄老师可真不是乌鸦嘴，这种 RAID 1 磁盘阵列中的硬盘设备同时损坏的情况还真被我的学生遇到过。

在这样的情况下，该怎么办呢？其实，我们完全可以使用 RAID 备份盘技术来预防这类事故。该技术的核心理念就是准备一块足够大的硬盘，这块硬盘平时处于闲置状态，一旦 RAID 磁盘阵列中有硬盘出现故障后则会马上自动顶替上去。这样很棒吧！

为了避免多个实验之间相互发生冲突，我们需要保证每个实验的相对独立性，为此需要大家自行将虚拟机还原到初始状态。另外，由于刚才已经演示了 RAID 10 磁盘阵列的部署方法，我们现在来看一下 RAID 5 的部署效果。部署 RAID 5 磁盘阵列时，至少需要用到 3 块硬盘，还需要再加一块备份硬盘，所以总计需要在虚拟机中模拟 4 块硬盘设备，如图 7-6 所示。

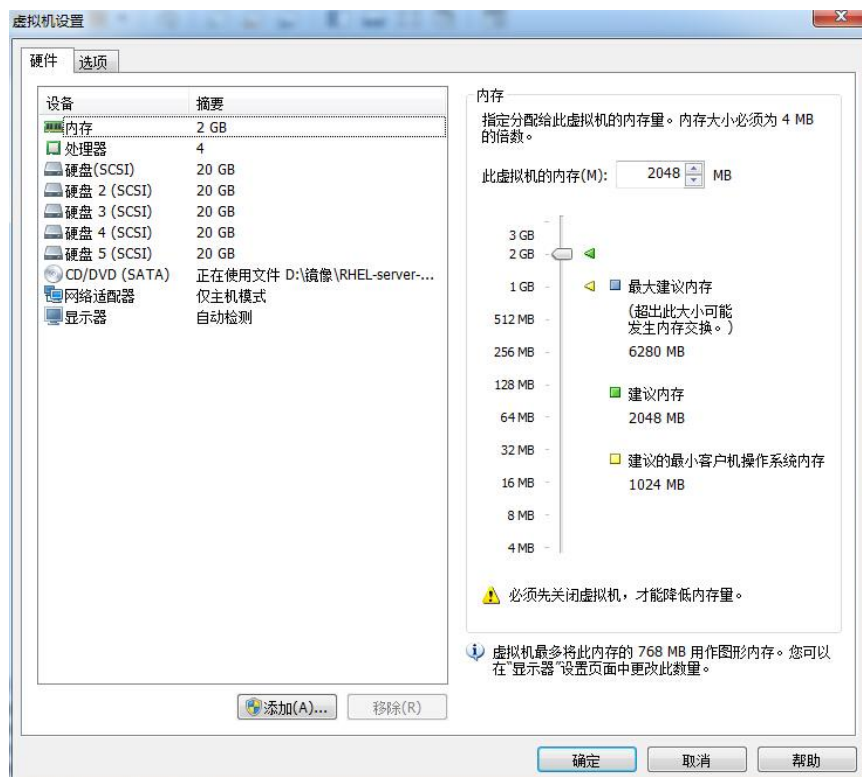


图 7-6 在虚拟机中模拟添加 4 块硬盘设备

现在创建一个 RAID 5 磁盘阵列+备份盘。在下面的命令中，参数-n 3 代表创建这个 RAID 5 磁盘阵列所需的硬盘数，参数-l 5 代表 RAID 的级别，而参数-x 1 则代表有一块备份盘。当查看/dev/md0（即 RAID 5 磁盘阵列的名称）磁盘阵列的时候就能看到有一块备份盘在等待中了。

```
[root@linuxprobe ~]# mdadm -Cv /dev/md0 -n 3 -l 5 -x 1 /dev/sdb /dev/sdc /dev/
sdd /dev/sde
mdadm: layout defaults to left-symmetric
mdadm: layout defaults to left-symmetric
mdadm: chunk size defaults to 512K
mdadm: size set to 20954624K
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
[root@linuxprobe ~]# mdadm -D /dev/md0
/dev/md0:
Version : 1.2
Creation Time : Fri May 8 09:20:35 2017
Raid Level : raid5
Array Size : 41909248 (39.97 GiB 42.92 GB)
Used Dev Size : 20954624 (19.98 GiB 21.46 GB)
Raid Devices : 3
Total Devices : 4
Persistence : Superblock is persistent
Update Time : Fri May 8 09:22:22 2017
State : clean
Active Devices : 3
```

```
Working Devices : 4
Failed Devices : 0
Spare Devices : 1
Layout : left-symmetric
Chunk Size : 512K
Name : linuxprobe.com:0 (local to host linuxprobe.com)
UUID : 44b1a152:3f1809d3:1d234916:4ac70481
Events : 18
Number Major Minor RaidDevice State
    0   8   16       0    active sync /dev/sdb
    1   8   32       1    active sync /dev/sdc
    4   8   48       2    active sync /dev/sdd
    3   8   64       -    spare     /dev/sde
```

现在将部署好的 RAID 5 磁盘阵列格式化为 ext4 文件格式，然后挂载到目录上，之后就可以使用了。

```
[root@linuxprobe ~]# mkfs.ext4 /dev/md0
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=128 blocks, Stripe width=256 blocks
2621440 inodes, 10477312 blocks
523865 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=2157969408
320 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
4096000, 7962624
Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done
[root@linuxprobe ~]# echo "/dev/md0 /RAID ext4 defaults 0 0" >> /etc/fstab
[root@linuxprobe ~]# mkdir /RAID
[root@linuxprobe ~]# mount -a
```

最后是见证奇迹的时刻！我们再次把硬盘设备/dev/sdb 移出磁盘阵列，然后迅速查看 /dev/md0 磁盘阵列的状态，就会发现备份盘已经被自动顶替上去并开始了数据同步。RAID 中的这种备份盘技术非常实用，可以在保证 RAID 磁盘阵列数据安全性的基础上进一步提高数据可靠性，所以，如果公司不差钱的话还是再买上一块备份盘以防万一。

```
[root@linuxprobe ~]# mdadm /dev/md0 -f /dev/sdb
mdadm: set /dev/sdb faulty in /dev/md0
[root@linuxprobe ~]# mdadm -D /dev/md0
/dev/md0:
Version : 1.2
Creation Time : Fri May 8 09:20:35 2017
Raid Level : raid5
Array Size : 41909248 (39.97 GiB 42.92 GB)
```

```
Used Dev Size : 20954624 (19.98 GiB 21.46 GB)
Raid Devices : 3
Total Devices : 4
Persistence : Superblock is persistent
Update Time : Fri May 8 09:23:51 2017
State : active, degraded, recovering
Active Devices : 2
Working Devices : 3
Failed Devices : 1
Spare Devices : 1
Layout : left-symmetric
Chunk Size : 512K
Rebuild Status : 0% complete
Name : linuxprobe.com:0 (local to host linuxprobe.com)
UUID : 44b1a152:3f1809d3:1d234916:4ac70481
Events : 21
Number Major Minor RaidDevice State
    3      8     64      0    spare rebuilding /dev/sde
    1      8     32      1    active sync      /dev/sdc
    4      8     48      2    active sync      /dev/sdd
    0      8     16      -    faulty         /dev/sdb
```

7.2 LVM（逻辑卷管理器）

前面学习的硬盘设备管理技术虽然能够有效地提高硬盘设备的读写速度以及数据的安全性，但是在硬盘分分区或者部署为 RAID 磁盘阵列之后，再想修改硬盘分区大小就不容易了。换句话说，当用户想要随着实际需求的变化调整硬盘分区的大小时，会受到硬盘“灵活性”的限制。这时就需要用到另外一项非常普及的硬盘设备资源管理技术了——LVM（逻辑卷管理器）。LVM 可以允许用户对硬盘资源进行动态调整。

逻辑卷管理器是 Linux 系统用于对硬盘分区进行管理的一种机制，理论性较强，其创建初衷是为了解决硬盘设备在创建分区后不易修改分区大小的缺陷。尽管对传统的硬盘分区进行强制扩容或缩容从理论上来讲是可行的，但是却可能造成数据的丢失。而 LVM 技术是在硬盘分区和文件系统之间添加了一个逻辑层，它提供了一个抽象的卷组，可以把多块硬盘进行卷组合并。这样一来，用户不必关心物理硬盘设备的底层架构和布局，就可以实现对硬盘分区的动态调整。LVM 的技术架构如图 7-7 所示。

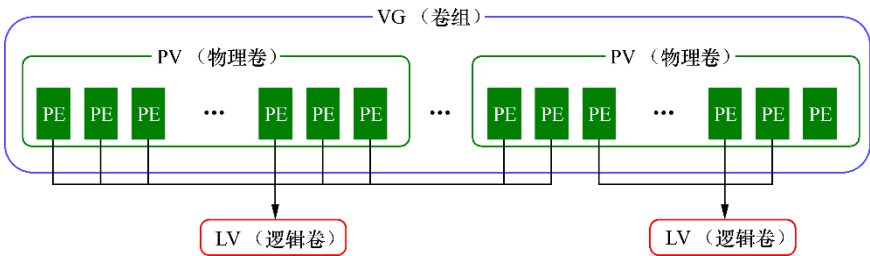


图 7-7 逻辑卷管理器的技术结构

为了帮助大家理解，刘遑老师来举一个吃货的例子。比如小明家里想吃馒头但是面粉不

够了，于是妈妈从隔壁老王家、老李家、老张家分别借来一些面粉，准备蒸馒头吃。首先需要把这些面粉（物理卷[PV, Physical Volume]）揉成一个大面团（卷组[VG, Volume Group]），然后再把这个大面团分割成一个个小馒头（逻辑卷[LV, Logical Volume]），而且每个小馒头的重量必须是每勺面粉（基本单元[PE, Physical Extent]）的倍数。

物理卷处于 LVM 中的最底层，可以将其理解为物理硬盘、硬盘分区或者 RAID 磁盘阵列，这都可以。卷组建立在物理卷之上，一个卷组可以包含多个物理卷，而且在卷组创建之后也可以继续向其中添加新的物理卷。逻辑卷是用卷组中空闲的资源建立的，并且逻辑卷在建立后可以动态地扩展或缩小空间。这就是 LVM 的核心理念。

7.2.1 部署逻辑卷

一般而言，在生产环境中无法精确地评估每个硬盘分区在日后的使用情况，因此会导致原先分配的硬盘分区不够用。比如，伴随着业务量的增加，用于存放交易记录的数据库目录的体积也随之增加；因为分析并记录用户的行为从而导致日志目录的体积不断变大，这些都会导致原有的硬盘分区在使用上捉襟见肘。而且，还存在对较大的硬盘分区进行精简缩容的情况。

我们可以通过部署 LVM 来解决上述问题。部署 LVM 时，需要逐个配置物理卷、卷组和逻辑卷。常用的部署命令如表 7-2 所示。

表 7-2 常用的 LVM 部署命令

功能/命令	物理卷管理	卷组管理	逻辑卷管理
扫描	<code>pvscan</code>	<code>vgscan</code>	<code>lvscan</code>
建立	<code>pvccreate</code>	<code>vgcreate</code>	<code>lvcreate</code>
显示	<code>pvdisplay</code>	<code>vgdisplay</code>	<code>lvdisplay</code>
删除	<code>pvremove</code>	<code>vgremove</code>	<code>lvremove</code>
扩展		<code>vgextend</code>	<code>lvextend</code>
缩小		<code>vgreduce</code>	<code>lvreduce</code>

为了避免多个实验之间相互发生冲突，请大家自行将虚拟机还原到初始状态，并在虚拟机中添加两块新硬盘设备，然后开机，如图 7-8 所示。

在虚拟机中添加两块新硬盘设备的目的，是为了更好地演示 LVM 理念中用户无需关心底层物理硬盘设备的特性。我们先对这两块新硬盘进行创建物理卷的操作，可以将该操作简单理解成让硬盘设备支持 LVM 技术，或者理解成是把硬盘设备加入到 LVM 技术可用的硬件资源池中，然后对这两块硬盘进行卷组合并，卷组的名称可以由用户来自定义。接下来，根据需求把合并后的卷组切割出一个约为 150MB 的逻辑卷设备，最后把这个逻辑卷设备格式化成 EXT4 文件系统后挂载使用。在下文中，刘遄老师将对每一个步骤再作一些简单的描述。

第 1 步：让新添加的两块硬盘设备支持 LVM 技术。

```
[root@linuxprobe ~]# pvccreate /dev/sdb /dev/sdc
Physical volume "/dev/sdb" successfully created
Physical volume "/dev/sdc" successfully created
```

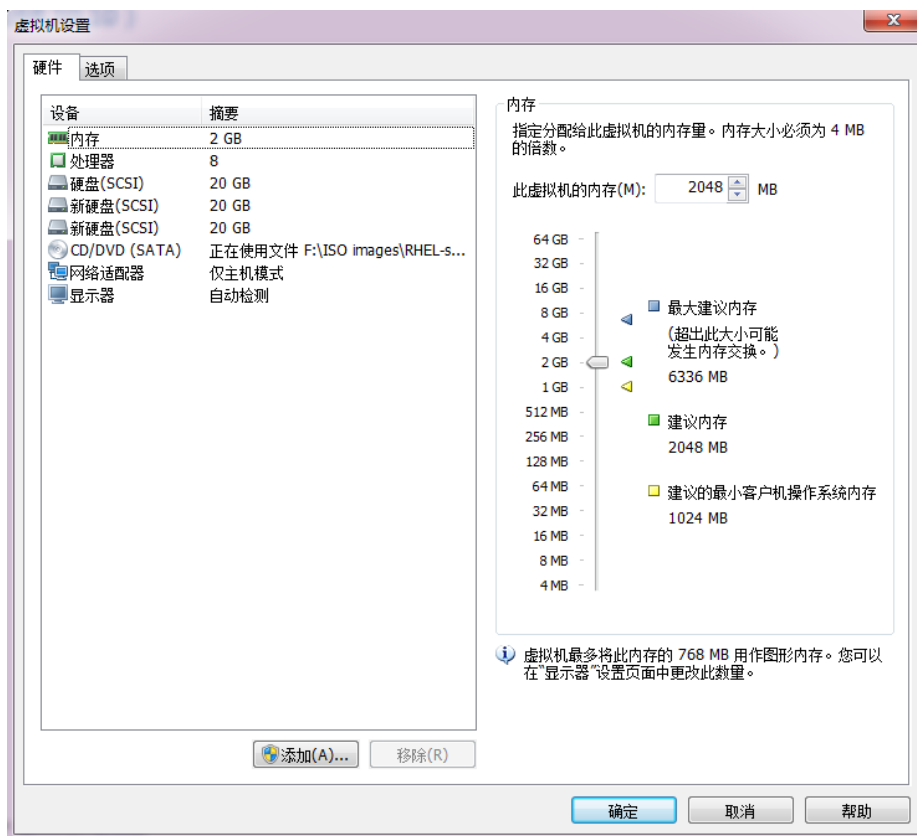


图 7-8 在虚拟机中添加两块新的硬盘设备

第 2 步：把两块硬盘设备加入到 storage 卷组中，然后查看卷组的状态。

```
[root@linuxprobe ~]# vgcreate storage /dev/sdb /dev/sdc
Volume group "storage" successfully created
[root@linuxprobe ~]# vgdisplay
--- Volume group ---
VG Name storage
System ID
Format lvm2
Metadata Areas 2
Metadata Sequence No 1
VG Access read/write
VG Status resizable
MAX LV 0
Cur LV 0
Open LV 0
Max PV 0
Cur PV 2
Act PV 2
VG Size 39.99 GiB
PE Size 4.00 MiB
Total PE 10238
Alloc PE / Size 0 / 0 Free PE / Size 10238 / 39.99 GiB
```

```
VG UUID KUEAMF-qMLh-XjQy-ArUo-LCQI-YF0o-pScxm1
.....省略部分输出信息.....
```

第3步：切割出一个约为 150MB 的逻辑卷设备。

这里需要注意切割单位的问题。在对逻辑卷进行切割时有两种计量单位。第一种是以容量为单位，所使用的参数为-L。例如，使用-L 150M 生成一个大小为 150MB 的逻辑卷。另外一种是以基本单元的个数为单位，所使用的参数为-l。每个基本单元的大小默认为 4MB。例如，使用-l 37 可以生成一个大小为 37×4MB=148MB 的逻辑卷。

```
[root@linuxprobe ~]# lvcreate -n vo -l 37 storage
Logical volume "vo" created
[root@linuxprobe ~]# lvdisplay
--- Logical volume ---
LV Path /dev/storage/vo
LV Name vo
VG Name storage
LV UUID D09HYI-BHB1-iXGr-X2n4-HEzo-FAQH-HRCM2I
LV Write Access read/write
LV Creation host, time localhost.localdomain, 2017-02-01 01:22:54 -0500
LV Status available
# open 0
LV Size 148.00 MiB
Current LE 37
Segments 1
Allocation inherit
Read ahead sectors auto
- currently set to 8192
Block device 253:2
.....省略部分输出信息.....
```

第4步：把生成好的逻辑卷进行格式化，然后挂载使用。

Linux 系统会把 LVM 中的逻辑卷设备存放在/dev 设备目录中（实际上是做了一个符号链接），同时会以卷组的名称来建立一个目录，其中保存了逻辑卷的设备映射文件（即/dev/卷组名称/逻辑卷名称）。

```
[root@linuxprobe ~]# mkfs.ext4 /dev/storage/vo
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
Stride=0 blocks, Stripe width=0 blocks
38000 inodes, 151552 blocks
7577 blocks (5.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=33816576
19 block groups
8192 blocks per group, 8192 fragments per group
2000 inodes per group
Superblock backups stored on blocks:
8193, 24577, 40961, 57345, 73729
Allocating group tables: done
```

```
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
[root@linuxprobe ~]# mkdir /linuxprobe
[root@linuxprobe ~]# mount /dev/storage/vo /linuxprobe
```

第 5 步：查看挂载状态，并写入到配置文件，使其永久生效。

```
[root@linuxprobe ~]# df -h
Filesystem Size Used Avail Use% Mounted on
/dev/mapper/rhel-root 18G 3.0G 15G 17% /
devtmpfs 905M 0 905M 0% /dev
tmpfs 914M 140K 914M 1% /dev/shm
tmpfs 914M 8.8M 905M 1% /run
tmpfs 914M 0 914M 0% /sys/fs/cgroup
/dev/sr0 3.5G 3.5G 0 100% /media/cdrom
/dev/sda1 497M 119M 379M 24% /boot
/dev/mapper/storage-vo 145M 7.6M 138M 6% /linuxprobe
[root@linuxprobe ~]# echo "/dev/storage/vo /linuxprobe ext4 defaults 0 0" >> /
etc/fstab
```

7.2.2 扩容逻辑卷

在前面的实验中，卷组是由两块硬盘设备共同组成的。用户在使用存储设备时感知不到设备底层的架构和布局，更不用关心底层是由多少块硬盘组成的，只要卷组中有足够的资源，就可以一直为逻辑卷扩容。扩展前请一定要记得卸载设备和挂载点的关联。

```
[root@linuxprobe ~]# umount /linuxprobe
```

第 1 步：把上一个实验中的逻辑卷 vo 扩展至 290MB。

```
[root@linuxprobe ~]# lvextend -L 290M /dev/storage/vo
Rounding size to boundary between physical extents: 292.00 MiB
Extending logical volume vo to 292.00 MiB
Logical volume vo successfully resized
```

第 2 步：检查硬盘完整性，并重置硬盘容量。

```
[root@linuxprobe ~]# e2fsck -f /dev/storage/vo
e2fsck 1.42.9 (28-Dec-2013)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
```

```
Pass 4: Checking reference counts
Pass 5: Checking group summary information
/dev/storage/vo: 11/38000 files (0.0% non-contiguous), 10453/151552 blocks
[root@linuxprobe ~]# resize2fs /dev/storage/vo
resize2fs 1.42.9 (28-Dec-2013)
Resizing the filesystem on /dev/storage/vo to 299008 (1k) blocks.
The filesystem on /dev/storage/vo is now 299008 blocks long.
```

第 3 步：重新挂载硬盘设备并查看挂载状态。

```
[root@linuxprobe ~]# mount -a
[root@linuxprobe ~]# df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/rhel-root	18G	3.0G	15G	17%	/
devtmpfs	985M	0	985M	0%	/dev
tmpfs	994M	80K	994M	1%	/dev/shm
tmpfs	994M	8.8M	986M	1%	/run
tmpfs	994M	0	994M	0%	/sys/fs/cgroup
/dev/sr0	3.5G	3.5G	0	100%	/media/cdrom
/dev/sda1	497M	119M	379M	24%	/boot
/dev/mapper/storage-vo	279M	2.1M	259M	1%	/linuxprobe

7.2.3 缩小逻辑卷

相较于扩容逻辑卷，在对逻辑卷进行缩容操作时，其丢失数据的风险更大。所以在生产环境中执行相应操作时，一定要提前备份好数据。另外 Linux 系统规定，在对 LVM 逻辑卷进行缩容操作之前，要先检查文件系统的完整性（当然这也是为了保证我们的数据安全）。在执行缩容操作前记得先把文件系统卸载掉。

```
[root@linuxprobe ~]# umount /linuxprobe
```

第 1 步：检查文件系统的完整性。

```
[root@linuxprobe ~]# e2fsck -f /dev/storage/vo
e2fsck 1.42.9 (28-Dec-2013)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
/dev/storage/vo: 11/74000 files (0.0% non-contiguous), 15507/299008 blocks
```

第 2 步：把逻辑卷 vo 的容量减小到 120MB。

```
[root@linuxprobe ~]# resize2fs /dev/storage/vo 120M
resize2fs 1.42.9 (28-Dec-2013)
Resizing the filesystem on /dev/storage/vo to 122880 (1k) blocks.
The filesystem on /dev/storage/vo is now 122880 blocks long.
[root@linuxprobe ~]# lvreduce -L 120M /dev/storage/vo
WARNING: Reducing active logical volume to 120.00 MiB
```

```
THIS MAY DESTROY YOUR DATA (filesystem etc.)
Do you really want to reduce vo? [y/n]: y
Reducing logical volume vo to 120.00 MiB
Logical volume vo successfully resized
```

第 3 步：重新挂载文件系统并查看系统状态。

```
[root@linuxprobe ~]# mount -a
[root@linuxprobe ~]# df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/rhel-root	18G	3.0G	15G	17%	/
devtmpfs	985M	0	985M	0%	/dev
tmpfs	994M	80K	994M	1%	/dev/shm
tmpfs	994M	8.8M	986M	1%	/run
tmpfs	994M	0	994M	0%	/sys/fs/cgroup
/dev/sr0	3.5G	3.5G	0	100%	/media/cdrom
/dev/sda1	497M	119M	379M	24%	/boot
/dev/mapper/storage-vo	113M	1.6M	103M	2%	/linuxprobe

7.2.4 逻辑卷快照

LVM 还具备有“快照卷”功能，该功能类似于虚拟机软件的还原时间点功能。例如，可以对某一个逻辑卷设备做一次快照，如果日后发现数据被改错了，就可以利用之前做好的快照卷进行覆盖还原。LVM 的快照卷功能有两个特点：

- 快照卷的容量必须等同于逻辑卷的容量；
- 快照卷仅一次有效，一旦执行还原操作后则会被立即自动删除。

首先查看卷组的信息。

```
[root@linuxprobe ~]# vgdisplay
--- Volume group ---
VG Name storage
System ID
Format lvm2
Metadata Areas 2
Metadata Sequence No 4
VG Access read/write
VG Status resizable
MAX LV 0
Cur LV 1
Open LV 1
Max PV 0
Cur PV 2
Act PV 2
VG Size 39.99 GiB
PE Size 4.00 MiB
Total PE 10238
Alloc PE / Size 30 / 120.00 MiB Free PE / Size 10208 / 39.88 GiB
VG UUID CTaHAK-0TQv-Abdb-R83O-RU6V-YYkx-8o2R0e
.....省略部分输出信息.....
```

通过卷组的输出信息可以清晰看到，卷组中已经使用了 120MB 的容量，空闲容量还有

39.88GB。接下来用重定向逻辑卷设备所挂载的目录中写入一个文件。

```
[root@linuxprobe ~]# echo "Welcome to Linuxprobe.com" > /linuxprobe/readme.txt
[root@linuxprobe ~]# ls -l /linuxprobe
total 14
drwx-----. 2 root root 12288 Feb 1 07:18 lost+found
-rw-r--r--. 1 root root    26 Feb 1 07:38 readme.txt
```

第 1 步：使用-s 参数生成一个快照卷，使用-L 参数指定切割的大小。另外，还需要在命令后面写上是针对哪个逻辑卷执行的快照操作。

```
[root@linuxprobe ~]# lvcreate -L 120M -s -n SNAP /dev/storage/vo
Logical volume "SNAP" created
[root@linuxprobe ~]# lvdisplay
--- Logical volume ---
LV Path /dev/storage/SNAP
LV Name SNAP
VG Name storage
LV UUID BC7WKg-fHoK-Pc7J-yhSd-vD7d-lUnl-TihKlt
LV Write Access read/write
LV Creation host, time localhost.localdomain, 2017-02-01 07:42:31 -0500
LV snapshot status active destination for vo
LV Status available
# open 0
LV Size 120.00 MiB
Current LE 30
COW-table size 120.00 MiB
COW-table LE 30
Allocated to snapshot 0.01%
Snapshot chunk size 4.00 KiB
Segments 1
Allocation inherit
Read ahead sectors auto
- currently set to 8192
Block device 253:3
.....省略部分输出信息.....
```

第 2 步：在逻辑卷所挂载的目录中创建一个 100MB 的垃圾文件，然后再查看快照卷的状态。可以发现存储空间占的用量上升了。

```
[root@linuxprobe ~]# dd if=/dev/zero of=/linuxprobe/files count=1 bs=100M
1+0 records in
1+0 records out
104857600 bytes (105 MB) copied, 3.35432 s, 31.3 MB/s
[root@linuxprobe ~]# lvdisplay
--- Logical volume ---
LV Path /dev/storage/SNAP
LV Name SNAP
VG Name storage
LV UUID BC7WKg-fHoK-Pc7J-yhSd-vD7d-lUnl-TihKlt
LV Write Access read/write
LV Creation host, time localhost.localdomain, 2017-02-01 07:42:31 -0500
```

```
LV snapshot status active destination for vo
LV Status available
# open 0
LV Size 120.00 MiB
Current LE 30
COW-table size 120.00 MiB
COW-table LE 30
Allocated to snapshot 83.71%
Snapshot chunk size 4.00 KiB
Segments 1
Allocation inherit
Read ahead sectors auto
- currently set to 8192
Block device 253:3
```

第 3 步：为了校验 SNAP 快照卷的效果，需要对逻辑卷进行快照还原操作。在此之前记得先卸载掉逻辑卷设备与目录的挂载。

```
[root@linuxprobe ~]# umount /linuxprobe
[root@linuxprobe ~]# lvconvert --merge /dev/storage/SNAP
Merging of volume SNAP started.
vo: Merged: 21.4%
vo: Merged: 100.0%
Merge of snapshot into logical volume vo has finished.
Logical volume "SNAP" successfully removed
```

第 4 步：快照卷会被自动删除掉，并且刚刚在逻辑卷设备被执行快照操作后再创建出来的 100MB 的垃圾文件也被清除了。

```
[root@linuxprobe ~]# mount -a
[root@linuxprobe ~]# ls /linuxprobe/
lost+found readme.txt
```

7.2.5 删除逻辑卷

当生产环境中想要重新部署 LVM 或者不再需要使用 LVM 时，则需要执行 LVM 的删除操作。为此，需要提前备份好重要的数据信息，然后依次删除逻辑卷、卷组、物理卷设备，这个顺序不可颠倒。

第 1 步：取消逻辑卷与目录的挂载关联，删除配置文件中永久生效的设备参数。

```
[root@linuxprobe ~]# umount /linuxprobe
[root@linuxprobe ~]# vim /etc/fstab
#
# /etc/fstab
# Created by anaconda on Fri Feb 19 22:08:59 2017
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
```

```
#
/dev/mapper/rhel-root                /                xfs             defaults 1 1
UUID=50591e35-d47a-4aeb-a0ca-1b4e8336d9b1 /boot            xfs             defaults 1 2
/dev/mapper                          /rhel-swap       swap swap       defaults 0 0
/dev/cdrom                          /media/cdrom     iso9660         defaults 0 0
dev/storage/vo /linuxprobe ext4 defaults 0 0
```

第 2 步：删除逻辑卷设备，需要输入 y 来确认操作。

```
[root@linuxprobe ~]# lvremove /dev/storage/vo
Do you really want to remove active logical volume vo? [y/n]: y
Logical volume "vo" successfully removed
```

第 3 步：删除卷组，此处只写卷组名称即可，不需要设备的绝对路径。

```
[root@linuxprobe ~]# vgremove storage
Volume group "storage" successfully removed
```

第 4 步：删除物理卷设备。

```
[root@linuxprobe ~]# pvremove /dev/sdb /dev/sdc
Labels on physical volume "/dev/sdb" successfully wiped
Labels on physical volume "/dev/sdc" successfully wiped
```

在上述操作执行完毕之后，再执行 `lvdisplay`、`vgdisplay`、`pvdisplay` 命令来查看 LVM 的信息时就不会再看到信息了（前提是上述步骤的操作是正确的）。

复习题

1. RAID 技术主要是为了解决什么问题呢？

答：RAID 技术可以解决存储设备的读写速度问题及数据的冗余备份问题。

2. RAID 0 和 RAID 5 哪个更安全？

答：RAID 0 没有数据冗余功能，因此 RAID 5 更安全。

3. 假设使用 4 块硬盘来部署 RAID 10 方案，外加一块备份盘，最多可以允许几块硬盘同时损坏呢？

答：最多允许 5 块硬盘设备中的 3 块设备同时损坏。

4. 位于 LVM 最底层的是物理卷还是卷组？

答：最底层的是物理卷，然后在通过物理卷组成卷组。

5. LVM 对逻辑卷的扩容和缩容操作有何异同点呢？

答：扩容和缩容操作都需要先取消逻辑卷与目录的挂载关联；扩容操作是先扩容后检查文件系统完整性，而缩容操作为了保证数据的安全，需要先检查文件系统完整性再缩容。

6. LVM 的快照卷能使用几次？

答：只可使用一次，而且使用后即自动删除。

7. LVM 的删除顺序是怎么样的？

答：依次移除逻辑卷、卷组和物理卷。